1. Technical Debt Diagnosis

1.1 Identify the hidden-technical-debt categories

(a) **Entanglement** — the weights are fit together, so the coupling is in the model, not the code; nothing links the two features. Rounding delivery time threw away signal, the optimiser made it up elsewhere, and favourite-restaurants ended up standing in for it. Nothing broke, the recommendations just got worse.

(b) **Undeclared Consumers** — marketing started reading the output table without telling anyone, so the ML team's dependency graph is wrong. A schema tweak, a rescaling, or a routine retrain all look safe from the inside, and any of them can break a campaign the team doesn't know about.

(c) **Configuration & Glue-Code Debt** — the 14 chained scripts are the pipeline jungle: extraction, transformation and model calls taped together with no orchestration layer, hyperparameters sitting inside the scripts. The manual steps were never written down, so nobody can reproduce the setup off the author's machine.

1.2 Mitigation for (c)

Tool: MLflow Projects - an `MLproject` file with declared entry points, a `python_env.yaml`, run via `mlflow run`.

- Swap the 14 scripts for one `MLproject` file with named entry points (`main`, `evaluate`), typed parameters and defaults, and a `python_env.yaml` pinning the interpreter and library versions.
- **Configuration Debt:** hyperparameters leave the shell scripts and become declared parameters with one default, overridden at the command line via `mlflow run . -P n_estimators=300`.
- **Glue-Code Debt:** the chain becomes one documented command that runs locally or from a Git URL. Each `mlflow run` opens a Tracking run, logging params, metrics and the `git_commit` tag. That is the run pinned.

2. MLflow Experiment Comparison (MLP on MNIST)

Six runs, `lr` $\in \{0.001, 0.01, 0.1\} \times$ `batch_size` $\in \{32, 256\}$. Fixed: 128 hidden units, 30 epochs, Adam, seed 42, 10k train / 2k val.

lr	bs	final_train_loss	final_val_acc	best_val_acc	overfit_gap
0.001	32	0.0053	**0.9525**	0.9545	0.0463
0.001	256	0.0231	0.9470	0.9470	0.0492
0.01	256	0.0288	0.9465	0.9530	0.0440
0.01	32	0.0932	0.9275	0.9420	0.0466
0.1	256	0.5387	0.8160	0.8795	0.0244
0.1	32	1.0876	0.6900	0.8010	0.0136

- **Best run:** `lr=0.001, bs=32` at 0.9525 val accuracy and the lowest train loss (0.0053). Small enough for Adam to stay stable yet still converging inside 30 epochs; at `lr=0.1` it overshoots and never settles. The small batch helps at the margin — 313 updates per epoch against 40 at `bs=256`.
- **Overfitting:** after roughly epoch 8 `train_loss` keeps falling 0.040 → 0.005 (8×) while `val_accuracy` flattens in the 0.945–0.952 band and merely wobbles. Final train accuracy 0.9988 against 0.9525 validation, a gap of 0.0463 — the extra epochs buy memorisation, not generalisation.
- **Larger effect: learning rate.** Its effect is 0.2625 (max − min over all six runs) against 0.0502 for batch size (mean $|\Delta|$ within each `lr` pair), so `lr` dominates by 5.2×. The parallel-coordinates plot agrees — lines fan out over `learning_rate`, collapse over `batch_size`.

3. DVC Data Versioning & Rollback

Cats/dogs dataset tracked with DVC, pushed to an S3 remote (`s3://aiops-kiran-a1q3/q3-dvc`); only `.dvc` pointers go into Git, never the images.

- **v1** (`8bb78c1`) = 1800 images, `labels.csv` 1801 lines. **v2** (`ffbd981`) = 2800 images after adding 1000 more, `labels.csv` 2801 lines. Both are Git tags.
- The rollback transcript proves the seperation of code and data: `git checkout v1` alone moves *pointers only* — `labels.csv` is still sitting at 2801 lines and `dvc diff` lists 1000 files as added. Only after `dvc checkout` does the workspace snap back to 1800 images / 1801 lines, and `dvc status` then reports up to date.
- `git checkout main && dvc checkout` restores v2 exactly — Git versions the pointer, DVC materialises the bytes, and the two steps are independant.

4. End-to-End Reproducibility Capstone

One MLP run on MNIST versioned across all three layers at once — code in Git, data in DVC/S3, metrics and model in MLflow. Partner A trained (`lr=1e-3`, hidden `(128,128)`, `alpha=1e-4`, `max_iter=100`, early stopping, seed 42) and registered `mnist-mlp` in stage `Staging`, with `run_id` and `git_commit` as run tags. Partner B rebuilt from a clean clone only: checkout the commit, `dvc pull`, venv from the pinned `requirements.txt`, re-run the notebook.

	Partner A	Partner B
Accuracy	0.9771428571428571	0.9771428571428571
Macro F1	0.9769855553603207	0.9769855553603207
Epochs to converge	16	16
`git_commit`	`daa1875`	`daa1875`

- **Verdict: MATCHED** — delta **+0.0000** against a tolerance of ±0.005, agreeing to all sixteen significant figures. Enviroment identical on both sides (Python 3.14.4, scikit-learn 1.9.0, numpy 2.5.2, mlflow 3.15.2), logged per run to `run_environment.json`.
- Partner B's run carries `reproduction_verdict` / `reproduction_delta` tags and a `reproduction_note.txt` artifact. Only deviation: `register_model` was not re-run — that is Partner A's step, and it would of added a junk version.